

A1 production kernel manifest – imports the ORIGINAL N-001 solver; enforces eta_shell=0 (v1.4)

TECT verification-first repository · claim A1-PRODUCTION-KERNEL-MANIFEST

first issued 2026-06-23 · this version issued 2026-06-23 · v1.4

1. Purpose and scope

A config is the numerical implementation of A2/A3 only if it is the kernel actually implemented by the canonical N-001 microscopic PDE solver. v1.3 checked a vendored mimic that omitted the solver’s shell-bias term; v1.4 imports the ORIGINAL solver functions and enforces the pure-Brazovskii condition. Verification: [codes/foundations/a1_kernel_checks.py](#) v1.4 (13/13 PASS); solver sources [codes/foundations/n001_solver/](#); config [claims/A1-PRODUCTION-KERNEL-MANIFEST/canonical_n001_kernel.json](#).

2. The real N-001 symbol and the eta_shell=0 slice

The N-001 solver’s linear operator (`bloch_matrix_linear` in `bloch_linearization.py`) has scalar diagonal

$$D_{\text{solver}}(k) = r + Z|k|^2 + Y|k|^4 + \eta_{\text{shell}} (|k| - q_0)^2, \quad (r, Z) = \text{kinetic_coefficients}(\text{mu2_shell}, Y, q_0). \quad (1)$$

The pure Brazovskii kernel certified by A1/A2/A3 is the $\eta_{\text{shell}} = 0$ slice. The config stores $\eta_{\text{shell}} = 0$ as a sixth physical field; the verifier ENFORCES $\eta_{\text{shell}} = 0$ and confirms the ORIGINAL `bloch_matrix_linear`($\eta_{\text{shell}} = 0$) reduces to $r_{\text{zero}} + Z|k|^2 + Y|k|^4$ = the stored kernel (8.9×10^{-16}). The shell-bias term is load-bearing: setting $\eta_{\text{shell}} = 0.1$ shifts the ORIGINAL symbol by up to 0.231, so $\eta_{\text{shell}} = 0$ is an essential, non-vacuous condition.

3. Original-source provenance (v1.4)

Three solver sources are vendored byte-identical from the frozen TECT2 tree and imported directly (full sha256 pinned and checked in `n001_solver/PROVENANCE.json` and the config): `continuation_mu2_v25.py` (6db0393a..), `bloch_linearization.py` (337bd6de..), `math56_constants.py` (6c808597.., a transitive import dependency). The verifier confirms `kinetic_coefficients(mu2_shell, Y, q_0) = (r_zero, Z)` and detects a corrupted stored q_0 (+1% $\Rightarrow O(10^{-2})$ divergence).

4. Status: gates pass, certification pending

Under $\eta_{\text{shell}} = 0$ the canonical N-001 config clears both manifests ($\delta_Z = \delta_r = 0$, $\delta_m = 4.3 \times 10^{-18}$, $\text{mu2_shell} > 0$) AND matches the ORIGINAL solver symbol. The script reports “numerical gates pass; operator certification pending”: the card stays T1 OPEN pending the operator’s independent run of the bundle (config + the three solver sources + the checker).

5. Devil’s-advocate

(α) “ $\eta_{\text{shell}} = 0$ assumed away.” DISMISSED — it is enforced AND shown load-bearing (0.231 shift at $\eta_{\text{shell}} = 0.1$); the certification is explicitly scoped to the pure-Brazovskii slice. (β) “*Vendored copies not the originals.*” DISMISSED — byte-identical (full sha256 pinned; compare to TECT2/Contents); the verifier imports them directly. (γ) “*Gates pass = certified.*” DISMISSED — both the script and this note state certification is pending; T1 OPEN retained. Quantitative sanity check: bloch symbol residual 8.9×10^{-16} ; shell-bias shift 0.231 at $\eta_{\text{shell}} = 0.1$.

6. Result footer

Result ID:	A1-PRODUCTION-KERNEL-MANIFEST (OPEN, v1.4)
Precise statement:	Verifier reads 6 stored fields (r_zero, mu2_shell, q0, Z, Y, eta_shell=0) AS-IS; imports the ORIGINAL N-001 solver (continuation_mu2_v25.kinetic_coefficients + bloch_linearization.bloch_matrix_linear): enforces eta_shell=0, confirms kinetic_coefficients(mu2_shell,Y, q0)=(r_zero,Z) and the original bloch symbol (eta_shell=0) = r_zero+Z k ^2+Y k ^4 (8.9e-16); shell-bias shown load-bearing; full sha256 of 3 sources pinned.
Scope:	pure-Brazovskii (eta_shell=0) scalar kernel; CONSISTENCY + ANALYTIC-BRANCH.
Dependencies:	A1-KERNEL-IDENTITY; A1-SCALAR-ANALYTIC-BRANCH.
Bundle:	canonical_n001_kernel.json + continuation_mu2_v25.py + bloch_linearization.py (+ math56_constants.py dep) + a1_kernel_checks.py.
Evidence grade:	ANALYTIC + EXECUTED (13/13) -- pending operator independent run.
Reproduction command:	python codes/foundations/a1_kernel_checks.py
Expected output:	13/13 PASS; eta_shell=0 enforced; original solver symbol matches; shell-bias load-bearing; sha256 pinned.
Tier:	T1 OPEN (gates pass; certification pending).
Falsifier:	eta_shell != 0, a source-hash mismatch, a stored-field edit breaking a delta, or kinetic_coefficients not reproducing the stored (r_zero,Z).
No-overclaim:	pure-Brazovskii eta_shell=0 slice only (full anisotropic operator excluded); gates passing != certification; not yet operator-certified, not yet A2/A3-citable.
Next required action:	Operator independently runs the bundle (13/13) and certifies the N-001 ANALYTIC-BRANCH manifest.